SYSTEM DESIGN MASTERY

Complete Curriculum · Templates · Tricks · Date-wise Schedule

Kashyap Pandey | Principal / Staff SDE Prep | March – May 2026

OVERVIEW — What This Document Covers

This is your single System Design reference. It covers the complete curriculum aligned with your 3-month prep plan, the universal HLD and LLD templates, the 8 building blocks, all memory techniques, and the date-wise study schedule. Read this before every Saturday session.

SCOPE	HLD + LLD + Foundations + Real-World Case Studies + Date-wise Schedule
Timeline	March 1 – May 31, 2026 (13 Saturday sessions, 10 systems)
Session Format	5 hrs every Saturday — 6-part structure per session
Target Companies	Google, Meta, Amazon, Uber, eBay, Visa, Amex
Target Role	Principal / Staff Software Engineer
Systems Covered	10 complete systems across 3 months
Alignment	100% aligned with your existing 3-Month Dated Prep Plan — no changes to schedule

THE 4 PILLARS — Every Saturday Session Covers All 4

Every system design session is built on exactly 4 pillars. You never skip one. Together they build Principal-level depth — not just interview knowledge.

PILLAR	WHAT IT BUILDS
Pillar 1: FOUNDATIONS	Concepts that never change — CAP theorem, latency numbers, DB internals, networking. Taught in context of each system, never as isolated boring theory.
Pillar 2: HLD (High Level Design)	Boxes and arrows. Which components, how they connect, why each exists. Uses the Universal HLD Template (7 steps). Covers distributed systems thinking.
Pillar 3: LLD (Low Level Design)	Classes, interfaces, methods, design patterns. You write real Java code for the 3 most critical classes of every system. Covers OOP mastery.
Pillar 4: REAL SYSTEMS	How actual companies solved this exact problem. Real war stories create emotional memory. You reference these in interviews AND in your actual Principal role.

THE 8 BUILDING BLOCKS — Your Universal Toolbox

Every distributed system ever built is a different combination of these 8 blocks. Learn them once. Apply them to every system forever. When you see a problem, you ask: which of these 8 do I need?

MEMORY TRICK	*"Load Databases Carefully. Make Cool APIs Often. Search Everything." — Load Balancer, Database, Cache, Message Queue, CDN, API Gateway, Object Storage, Search Index*

1 Load Balancer

WHAT: Distributes incoming traffic evenly across multiple servers so no single server is overwhelmed.
WHEN: Any system with more than 1 server. Always present in production. First thing you add when scaling.
TYPES: Round Robin | Least Connections | IP Hash (sticky sessions) | L4 (TCP) vs L7 (HTTP-aware)

2 Database

WHAT: Stores data permanently. The single source of truth for your system.
WHEN: Always. Every system needs persistent storage. The critical question is: which type and why?
TYPES: SQL: PostgreSQL, MySQL (relations, ACID, joins) | NoSQL: Cassandra, MongoDB (scale, flexibility)

3 Cache

WHAT: Stores frequently accessed hot data in memory for ultra-fast reads. Sits in front of the database.
WHEN: Read-heavy systems. When DB latency exceeds your SLA. When same data is read repeatedly.
TYPES: Redis (rich types, persistence, pub/sub) | Memcached (simple, fast) | Write-through vs Write-back

4 Message Queue

WHAT: Decouples services asynchronously. Producer sends messages. Consumer processes at its own pace.
WHEN: High write load. Background jobs. Fan-out to multiple consumers. When user should not wait.
TYPES: Kafka (high throughput, replay, your strength at Wells Fargo) | RabbitMQ (simplicity) | SQS (managed)

5 CDN (Content Delivery Network)

WHAT: Serves static content from edge servers geographically close to the user. Reduces origin server load.
WHEN: Images, videos, JS, CSS. Any system with global users. Dramatically reduces latency for static assets.
TYPES: Push CDN (you upload) | Pull CDN (fetches on first request) | Cloudflare, Akamai, AWS CloudFront

6 API Gateway

WHAT: Single entry point for all clients. Handles authentication, routing, rate limiting, SSL termination.

WHEN: Microservices architecture. Public APIs. When multiple services need unified, secure access control.
TYPES: Kong | AWS API Gateway | Nginx | Handles: Auth, Rate Limiting, Routing, Logging, Transformation

7 Object Storage

WHAT: Stores unstructured data — files, images, videos, documents, ML models — at massive scale cheaply.
WHEN: Any system storing files. Profile pictures, documents, videos, backups, ML training datasets.
TYPES: Amazon S3 | Google Cloud Storage | Azure Blob Storage | Cheap, durable, infinitely scalable

8 Search Index

WHAT: Makes data searchable at scale — full-text search, fuzzy matching, ranking, faceted filtering.
WHEN: E-commerce product search. Social media search. Any 'search bar' feature serving millions of queries.
TYPES: Elasticsearch (most common) | Apache Solr | Typesense | Lucene is the underlying engine for most

THE 8 DESIGN PRINCIPLES — Answer 80% of WHY Questions

These 8 principles are your cheat codes. Every time an interviewer asks WHY — one of these is the answer. Memorise them. Quote them. They signal Principal-level thinking.

PRINCIPLE	USE IT WHEN YOU NEED TO JUSTIFY...
1. Single Responsibility	Why you split into separate services. 'Each service does ONE thing well and has one reason to change.'
2. Cache Aggressively	Why Redis is in the design. 'If data is read more than it is written, cache it. Reads need microseconds, not milliseconds.'
3. Async by Default	Why Kafka or a queue is in the design. 'If the user does not need to wait for it, use a queue.'
4. Shard Early	Why the database needs horizontal partitioning. 'Design for sharding before you need it — retrofitting is painful.'
5. Eventual Consistency is OK	Why you chose availability over strong consistency. 'A social feed tolerates 60 seconds stale. A bank balance does not.'
6. Design for Failure	Why you have replicas, retries, circuit breakers. 'Every component WILL fail. Design assuming it does.'
7. Measure Everything	Why you have metrics, tracing, alerting. 'You cannot fix what you cannot see. Observability is not optional.'
8. Start Simple, Scale Later	Why you did not over-engineer from day one. 'Do not build for 100M users when you have 1000. Premature optimisation kills.'

UNIVERSAL HLD TEMPLATE — 7 Steps for Every System Design

This exact template works for every system design problem you will ever face. Internalize it so deeply that you run through it automatically — even under interview pressure.

MEMORY TRICK	***"Really Serious APIs Don't Handle Bad Traffic" — Requirements, Scale, API Design, Data Model, HLD, Bottlenecks, Trade-offs***

STEP 1
5 min

REQUIREMENTS — Ask Before You Design

- Functional Requirements: What must the system DO? List 3-5 core features only. Not every edge case.
- Non-Functional Requirements: Scale? Latency target (p99)? Availability (99.9% vs 99.99%)? Consistency?
- Out of Scope: Explicitly state what you are NOT building. This shows clarity of thought.
- NEVER skip this step. Jumping to boxes without requirements = instant credibility loss at Principal level.
- TRICK: Ask 3 clarifying questions minimum before drawing anything. Interviewers reward this.

STEP 2
5 min

SCALE ESTIMATION — Make Your Numbers Real

- DAU (Daily Active Users): e.g. 500M registered, 10% active daily = 50M DAU.
- Requests per second: DAU × requests per user per day ÷ 86400 seconds. Identify read:write ratio.
- Storage per day: data size per event × events per day. Multiply by 5 years for total capacity.
- Bandwidth: average response size × requests per second for both ingress and egress.
- TRICK: Round aggressively. 86400 sec/day ≈ 100K. 50M users × 10 req = 500M req/day ≈ 5000 req/sec.
- WHY IT MATTERS: Your component choices (SQL vs NoSQL, cache size) flow from these numbers.

STEP 3
5 min

API DESIGN — Define the Contract First

- Define 3-5 core REST API endpoints. Use REST unless there is a specific reason for gRPC or GraphQL.
- Format: METHOD /resource → functionName(params) → returnType
- Example: POST /tweets → createTweet(userId, content, mediaIds[]) → TweetId
- Example: GET /feed → getFeed(userId, pageToken, limit) → List<Tweet>
- TRICK: Designing API first forces you to think about data flow BEFORE drawing boxes.
- Include pagination tokens, authentication headers, and error response formats.

STEP 4
5 min

DATA MODEL — Design Your Schema

- Identify 3-5 core entities (tables or collections). Name every field you will query on.
- Define relationships: one-to-many, many-to-many (junction table needed?), foreign keys.
- Choose DB type and JUSTIFY with data: SQL (ACID, joins, <1B rows) vs NoSQL (scale, >1B rows).
- Always mention indexes on fields used in WHERE clauses, ORDER BY, and JOIN conditions.
- TRICK: If you can draw an ER diagram naturally, use SQL. If rows exceed 1 billion, think NoSQL.

- Show your schema: Table name, key fields, indexes, estimated row size.

STEP 5
15 min

HIGH LEVEL DESIGN — Draw the System

- Start with the simplest path: Client → API Gateway → Load Balancer → Service → DB.
- Add each component ONLY when you name a specific problem it solves. No components for decoration.
- Draw read path and write path separately — they often need very different components.
- Pick from your toolbox of 8 blocks. Every block you add must be justified with a WHY.
- TRICK: Talk while drawing. 'I am adding Redis here because reads are 100x more frequent than writes.'
- Draw arrows with labels: REST, gRPC, async, WebSocket, batch. Direction matters.

STEP 6
10 min

DEEP DIVE — Go Inside the Most Interesting Parts

- Interviewer picks OR you pick the 2 most challenging or interesting components.
- Go deep: internal algorithms, data structures, concurrency handling, failure modes.
- Connect to your real experience when possible: 'At Wells Fargo on ICMP, our Kafka pipeline...'
- TRICK: Say 'Let me go deeper on X because it is the most technically interesting part.' Take control.
- This is where Principal-level thinking becomes visible. Senior describes WHAT. Principal explains WHY.

STEP 7
5 min

BOTTLENECKS + TRADE-OFFS — Show You Think Ahead

- Name the exact component that breaks first at 10x load and exactly why it breaks.
- Identify single points of failure. What has no redundancy? What is your disaster recovery plan?
- State trade-offs you consciously made: consistency vs availability, cost vs performance, simplicity vs scale.
- End with: 'The biggest risk in this design is X, and I would address it by Y with more time.'
- TRICK: Interviewers score trade-off awareness more than perfect designs. No design is perfect — own it.

UNIVERSAL LLD TEMPLATE — 6 Steps for Every OOP Design

LLD is where your 11 years of Java shine. Use this 6-step framework for every low level design problem — parking lots, chess, elevators, payment systems, and more.

MEMORY TRICK	***"Every Relationship Becomes A Pattern In Java" — Entities, Relationships, Behaviors, Pattern, Interfaces, Java Code***

STEP 1
3 min

IDENTIFY ENTITIES — Find the Nouns

- Read the problem statement. Circle or underline every NOUN — each becomes a class.
- Parking Lot → ParkingLot, Spot, Vehicle, Ticket, Gate, Payment, Driver
- Chess → Board, Piece, Player, Move, Game, Clock
- eBay Bidding → Auction, Bid, Item, Seller, Buyer, Payment, Notification
- TRICK: If you have more than 8 entities you are over-modelling. Merge or simplify.
- Mark which entities are 'core' (required) vs 'supporting' (nice to have).

STEP 2
3 min

IDENTIFY RELATIONSHIPS — IS-A vs HAS-A vs USES

- IS-A → Inheritance. Car IS-A Vehicle. ElectricCar IS-A Car. Use sparingly.
- HAS-A → Composition. ParkingLot HAS-A list of Spots. Auction HAS-A list of Bids.
- USES → Dependency. Gate USES PaymentProcessor. Does not own it — injected.
- TRICK: Prefer composition over inheritance (HAS-A over IS-A). More flexible, easier to test.
- Draw a simple class diagram with arrows before writing any code.

STEP 3
3 min

IDENTIFY BEHAVIORS — Find the Verbs

- Read the problem. Underline every VERB — each becomes a method on some class.
- Park, Pay, Search, Reserve, Cancel, Bid, Notify, Validate, Calculate, Expire
- For each verb ask: WHO is responsible for this action? Assign to the most logical owner class.
- TRICK: If a class has more than 7 methods it is doing too much. Split it into two classes.
- Single Responsibility Principle: one reason to change = one job = one class.

STEP 4
5 min

APPLY DESIGN PATTERN — Choose the Right Structure

- Strategy: Multiple interchangeable algorithms at runtime. (Payment: Credit/Cash/UPI strategies)
- Observer: One event notifies multiple subscribers. (Bid placed → notify all bidders via Observer)
- Factory: Create objects without specifying exact type. (VehicleFactory.create('SUV'))
- State: Object behaviour changes with its state. (Elevator: IDLE, MOVING_UP, MOVING_DOWN)
- Singleton: Exactly one instance globally. (DatabaseConnectionPool, ConfigurationManager)
- Command: Encapsulate request as object. (Undo/redo operations, job scheduling queue)
- TRICK: Name the pattern out loud in the interview. 'I am applying Strategy pattern here because...'

STEP 5
5 min

INTERFACES FIRST — Define Contracts Before Code

- Define the interface BEFORE the implementation. Principal-level engineers always think in contracts.
- interface PaymentProcessor { boolean process(double amount, String currency, String orderId); }
- interface NotificationChannel { void send(Notification n, User recipient) throws DeliveryException; }
- Implementations: CreditCardProcessor, UPIProcessor, SMSChannel, EmailChannel, PushChannel
- TRICK: If you can swap implementations without changing any caller code = excellent design.
- Show how Dependency Injection wires interfaces to implementations — no hardcoded 'new'.

STEP 6
25 min

WRITE KEY CLASSES IN JAVA — Show Production Quality

- Write 3 most important classes in full Java. Leave simpler ones as stubs with descriptive comments.
- Show enums for states: enum SpotStatus { AVAILABLE, OCCUPIED, RESERVED, MAINTENANCE }
- Show how classes wire together — constructor injection, service orchestration.
- Include: null checks, thread safety (synchronized / ReentrantLock) if concurrent access needed.
- TRICK: Write 1 interface + 1 implementation fully. Stub the rest. Shows both breadth and depth.
- Add Javadoc to your key interface methods. It signals production engineering discipline.

DESIGN PATTERNS QUICK REFERENCE — When to Use Each

PATTERN	USE WHEN...
Strategy	Multiple ways to perform the same operation, swappable at runtime. Payment methods (Credit/Cash/UPI). Sorting algorithms. Compression formats. Discount calculation rules.
Observer	One event must trigger reactions in multiple independent components. Bid placed notifies all bidders. Stock price change alerts all subscribers. Order status updates all listeners.
Factory	Need to create objects without knowing exact type at compile time. VehicleFactory.create('TRUCK'). NotificationFactory.create('SMS'). Hides construction complexity from callers.
Singleton	Exactly one instance must exist throughout the application lifetime. DB connection pool. Thread pool executor. Application configuration manager. Logger instance.
State	Object changes its behaviour based on its internal state. Elevator (IDLE, MOVING, DOOR_OPEN). Vending machine (IDLE, SELECTING, DISPENSING). Order (PLACED, SHIPPED, DELIVERED).
Command	Encapsulate a request as an object — enables undo/redo, queuing, logging. Text editor undo. Job scheduler. Transaction log replay. Remote operation execution.
Decorator	Add behaviour to an object at runtime without changing its class. LoggingWrapper. AuthenticationWrapper. CompressionWrapper. CachingWrapper around any service.

Builder	Construct complex objects step by step with optional fields. SQL QueryBuilder. HTTP Request builder. Pizza/meal configurator. Complex configuration objects.
Repository	Abstract all data access behind an interface, hiding SQL/NoSQL details from business logic. UserRepository, OrderRepository, ProductRepository. Enables easy testing with mocks.
Template Method	Define skeleton algorithm in base class, let subclasses fill in the steps. Report generation (same structure, different data). Data import pipelines (parse, validate, transform, save).

THE PRINCIPAL ENGINEER DIFFERENCE — WHY Not Just HOW

The gap between Senior and Principal is not more technical knowledge. It is the ability to reason about WHY, back every decision with evidence, and own the trade-offs consciously.

LEVEL	WHAT THEY SAY IN AN INTERVIEW
Senior Engineer	We will use PostgreSQL for the database.
Principal Engineer	We will use PostgreSQL because our data is relational with strong consistency requirements, our 10K req/sec scale is well within PostgreSQL's comfortable range, and our team already has deep PostgreSQL operational expertise which reduces risk. If write throughput exceeds 200K/sec in year 2, we would evaluate Cassandra — but premature migration has a high engineering cost.

Every decision must contain exactly 3 parts:

WHAT	We use Redis for the caching layer.
WHY	Read latency must be under 10ms. The database averages 50ms. Redis delivers sub-millisecond reads from memory.
TRADE-OFF	We accept eventual consistency — feed data can be up to 60 seconds stale. This is acceptable for a social feed but would never be acceptable for a bank balance.

The 5 Dimensions a Principal Engineer Evaluates for Every Decision:

- **Correctness —** Does it actually solve the stated problem? What are all the edge cases and failure modes?
- **Reliability —** What happens when a server crashes? What is the blast radius? What is MTTR?
- **Scalability —** What is the bottleneck at 10x load? At 100x? Where does the architecture fundamentally break?
- **Maintainability —** Can a new engineer understand this in one week? How difficult are deployments and rollbacks?
- **Cost —** What is the infrastructure cost? The engineering cost to build? The operational cost to run for 5 years?

MEMORY TECHNIQUES — Remember System Design For Life

Technique 1 — The Why Chain

Never accept a component name at face value. Always ask WHY 3 times. By the third WHY you reach physics or first principles — and you will never forget the concept.

- Why Redis? → Reads are slow from the database.
- Why slow from DB? → The database is disk-based. Redis lives entirely in RAM.
- Why does RAM vs disk matter? → Memory access is 250 microseconds. Disk is 10 milliseconds. 40x difference.
- Now you understand Redis from physics up. That understanding never leaves you.

Technique 2 — The Contrast Method

Every concept is remembered best against its opposite. Always study concepts in pairs — what it is, and what it is not.

CONCEPT A	CONCEPT B — The Opposite
SQL: relations, ACID, joins, consistent	NoSQL: scale, flexible schema, eventual consistency
Cache: speed, volatile, temporary	Database: durability, persistent, slower
Synchronous: user waits for response	Asynchronous: queue handles it, user continues
Strong consistency: bank balance	Eventual consistency: social media feed
Monolith: simple, coupled, easy to start	Microservices: complex, independently scalable, hard to operate
Fan-out on write: fast reads, slow writes	Fan-out on read: fast writes, slow reads
Push CDN: you control content timing	Pull CDN: edge fetches on first user request

Technique 3 — The Real Story Anchor

Every concept tied to a real company story creates emotional memory. You remember the story first — and the concept comes with it automatically.

- **Fan-out problem** → Twitter 2012: Obama tweets during election. Celebrity's single tweet fans out to 50M followers. Servers crash. Forces complete rethink of feed architecture.
- **Consistent hashing** → Amazon DynamoDB: naive modulo sharding caused 'hot partitions'. Consistent hashing with virtual nodes solved uneven load distribution permanently.
- **CAP theorem reality** → MongoDB 2011: chose consistency over availability. Many production users surprised by unexpected downtime during network partition events.
- **Microservices cost** → Netflix: moved from monolith to 700+ microservices. Gained scale but needed 50+ new operational tools. 'Microservices are not free.'
- **Idempotency** → Stripe: a network timeout caused double charges for thousands of customers in 2012. Led to their now-famous idempotency key design — every charge API call requires a client-generated unique key.

Technique 4 — Draw It Cold

After every Saturday session: close all notes, take a blank piece of paper, and draw the full system from memory. Label every component. Write the trade-off for every decision. If you can draw it cold, you own it. If you cannot, you found a gap to fix.

Technique 5 — The 5-Minute Explain

3 days after every session, explain the complete system design in 5 minutes with no notes. This is exactly what a real interview feels like. Do it out loud — not in your head. Record yourself if possible. The more you do this, the more natural and confident you become under pressure.

SATURDAY SESSION STRUCTURE — The 6-Part Formula

Every Saturday follows this exact 6-part structure for 5 hours. Every system. Every week. The consistency builds muscle memory for the interview format.

PART	CONTENT	TIME
Part 1: Real System	How does this system ACTUALLY work at a real company? Case study before designing anything.	30 min
Part 2: You Design HLD	I play interviewer. You drive all 7 steps of HLD. I ask questions only — no guidance.	45 min
Part 3: LLD Deep Dive	Pick 2 key components. Design classes, interfaces, patterns. Write Java code.	30 min
Part 4: Gap Analysis	What did you miss? What would a Principal say differently? Honest debrief, no judgment.	20 min
Part 5: Real Company Case	Compare your design with how the actual company solved it. What differs and why?	20 min
Part 6: Your ADR	Write a 1-page Architecture Decision Record. Pushed to GitHub docs/system-design/.	15 min

Total: 160 minutes of active session | The ADR you write becomes your portfolio on GitHub.

DATE-WISE SYSTEM DESIGN SCHEDULE — Fully Aligned with 3-Month Plan

Every date below is a Saturday session (5 hrs). Every Sunday includes a system design mock using the previous Saturday's system. This schedule does NOT change the existing 3-month plan — it is the system design layer within it.

MONTH 1 — MARCH 2026 | Foundation + First 4 Systems

Sat, Mar 7
Saturday — 5 hrs

SYSTEM 1: Rate Limiter

- Foundation taught: CAP theorem, consistency models, Redis data structures and internals
- HLD: Token Bucket algorithm + Redis counters + API Gateway + distributed rate limiting
- LLD: RateLimiter interface, TokenBucketLimiter, SlidingWindowLimiter, RateLimitConfig
- Real World: How Stripe rate limits their API — per-key, per-endpoint, with burst allowance
- ADR: Why Token Bucket over Fixed Window — burst handling and smooth traffic shaping
- Sunday Mar 8 Mock: Reproduce Rate Limiter cold — requirements discipline and estimation

Sat, Mar 14
Saturday — 5 hrs

SYSTEM 2: URL Shortener (TinyURL / Bitly)

- Foundation taught: Hashing (MD5, SHA256), Base62 encoding, B-tree DB indexing deep dive
- HLD: Hash service + SQL database + Redis cache + CDN + analytics tracking pipeline
- LLD: URLShortener, HashGenerator, CacheManager, AnalyticsTracker, ExpiryManager
- Real World: How TinyURL differs from Bitly — custom slugs, analytics dashboards, TTL
- ADR: Why Base62 over MD5 truncation — collision probability and URL length trade-off
- Sunday Mar 15 Mock: Reproduce URL Shortener cold — HLD fluency and DB justification

Sat, Mar 21
Saturday — 5 hrs

SYSTEM 3: Push Notification System

- Foundation taught: Message queues deep dive, fan-out patterns, at-least-once vs exactly-once delivery
- HLD: Kafka + priority queues + channel workers + deduplication + exponential backoff retry
- LLD: NotificationService, Channel interface, SMSAdapter, EmailAdapter, PushAdapter, RetryManager
- Real World: How Amazon SNS handles 10 billion+ notifications per day across 200+ countries
- ADR: Why Kafka over SQS for notification fan-out — replay capability and ordering guarantees
- Sunday Mar 22 Mock: Reproduce Notification System — queue design and fan-out patterns

Sat, Mar 28
Saturday — 5 hrs

SYSTEM 4: Distributed Key-Value Store (like Redis)

- Foundation taught: Consistent hashing, virtual nodes, replication factor, gossip protocol
- HLD: Partitioned KV store with replication, leader election, anti-entropy, vector clocks
- LLD: Node, ConsistentHashRing, StorageEngine, GossipManager, ReplicationCoordinator
- Real World: How Redis Cluster uses hash slots for partitioning and automatic failover
- ADR: Why consistent hashing over range-based sharding — hot key distribution and rebalancing
- Sunday Mar 29 Mock: Month 1 review — explain any 2 designs from scratch in 5 minutes each

MONTH 2 — APRIL 2026 | Advanced Systems at Scale

Sat, Apr 4
Saturday — 5 hrs

SYSTEM 5: Kafka / Distributed Message Queue

- Foundation taught: Log-structured storage, ISR replication, ZooKeeper vs KRaft consensus
- HLD: Partitions, consumer groups, offset management, log compaction, exactly-once semantics
- LLD: KafkaProducer, KafkaConsumer, Topic, Partition, ConsumerGroup, OffsetManager in Java
- Real World: Your ICMP Wells Fargo Kafka experience — 19 services, async imaging pipelines
- ADR: Why Kafka over RabbitMQ for ICMP — throughput at scale, replay for audit, retention
- Sunday Apr 5 Mock: Reproduce Kafka design cold — partitions and replication deep dive

Sat, Apr 11
Saturday — 5 hrs

SYSTEM 6: Uber / Ride Sharing Platform

- Foundation taught: Geo-indexing (QuadTree, H3 hexagonal grid), WebSockets, real-time architectures
- HLD: Location service + matching engine + pricing engine + trip manager + WebSocket server
- LLD: TripManager, MatchingEngine, GeoIndex, SurgePricingEngine, LocationTracker, DriverPool
- Real World: Uber's migration from PostGIS to H3 hex grid — 7x improvement in geo query speed
- ADR: Why WebSockets over long-polling for driver location — connection overhead and battery cost
- Sunday Apr 12 Mock: Reproduce Uber HLD + surprise WhatsApp design — full 45-min simulation

Sat, Apr 18
Saturday — 5 hrs

SYSTEM 7: Payment Processing System (Visa / Amex Focus)

- Foundation taught: Idempotency keys, 2-Phase Commit, Saga pattern, distributed transactions
- HLD: Auth service + ledger service + settlement engine + fraud detection + reconciliation
- LLD: PaymentProcessor, IdempotencyManager, SagaCoordinator, FraudDetector, LedgerEntry
- Real World: How Stripe handles exactly-once payments — idempotency keys at every service layer

- ADR: Saga pattern over 2PC for payment flow — availability vs strong consistency trade-off
- Sunday Apr 19 Mock: Reproduce Payment System — idempotency and Saga deep dive focus

Sat, Apr 25
Saturday — 5 hrs

SYSTEM 8: eBay Marketplace + Bidding Platform REVENGE SESSION

- Foundation taught: Elasticsearch internals, inventory locking, optimistic concurrency control
- HLD: Catalog + Elasticsearch + inventory service + auction engine + order service + notifications
- LLD: AuctionEngine, BidProcessor, InventoryManager, OrderSaga, CatalogSearch, BidValidator
- Real World: eBay's actual bidding architecture — bid serialisation via Kafka, WebSocket results
- ADR: Why Kafka queue for bid serialisation — prevents double-winning under concurrent load
- The Core Answer: Bids → Kafka → single consumer → DB with optimistic locking → WebSocket to all bidders
- Sunday Apr 26 Mock: Full Mock Loop #1 — Design Twitter Feed (45-min end-to-end simulation)

MONTH 3 — MAY 2026 | AI Systems + Company Mocks + Final

Sat, May 2
Saturday — 5 hrs

SYSTEM 9: AI Document Processor (Spring AI + RAG)

- Foundation taught: Vector embeddings, cosine similarity, chunking strategies, LLM limitations
- HLD: Ingestion pipeline + embedding service + PGVector store + RAG retrieval + LLM + response cache
- LLD: DocumentProcessor, EmbeddingService, VectorStoreClient, RAGPipeline, ResponseCache
- Real World: Your ICMP Spring AI integration at Wells Fargo — production context and numbers
- ADR: Why RAG over fine-tuning for document Q&A — cost, data freshness, explainability, control
- Sunday May 3 Mock: Design 'AI Copilot for internal banking queries' — RAG + caching + eval

Sat, May 9
Saturday — 5 hrs

SYSTEM 10: Real-Time Fraud Detection (Visa / Amex)

- Foundation taught: Stream processing (Kafka + Flink), feature stores, ML model serving patterns
- HLD: Kafka ingest + Flink CEP + ML scorer + rules engine + alert service + case management
- LLD: FraudDetector, FeatureExtractor, RulesEngine, MLModelClient, AlertManager, CaseTracker
- Real World: How Visa achieves sub-100ms fraud decisions on 65,000+ transactions per second
- ADR: Online feature store (Redis) vs offline (data warehouse) — latency vs feature richness
- Sunday May 10 Mock: Full Mock Loop #2 — Amazon Order Management System (45-min simulation)

Sat, May 16
Saturday — 5 hrs

SYSTEM DESIGN MARATHON — 3 Designs Back-to-Back

- Design 1 (90 min): Real-time Leaderboard System — Redis sorted sets, score update at scale
- Design 2 (90 min): Distributed Job Scheduler — cron at scale, exactly-once execution, DAGs
- Design 3 (60 min): Log Aggregation System — Fluentd + Kafka + Elasticsearch + Kibana
- Goal: Full HLD + key LLD component in 90 minutes cold, no notes
- Sunday May 17 Mock: Full Mock Loop #3 — Real-time Payment Gateway (Visa/Amex difficulty)

Sat, May 23
Saturday — 5 hrs

FINAL MOCK — Principal Engineer Full Simulation

- Round 1 (45 min): Surprise HLD — Design Slack (real-time messaging at massive scale)
- Round 2 (45 min): AI System Design — Add intelligent AI features to your ICMP platform
- Round 3 (45 min): LLD Deep — Design core classes for Slack's real-time messaging engine
- Round 4 (45 min): Behavioral — System design stories aligned to Leadership Principles
- Debrief (60 min): Full scoring across all rounds + readiness assessment + apply checklist
- Sunday May 24: GRADUATION DAY — final surprise design + application strategy session

QUICK REFERENCE CARD — Read This Before Every Saturday Session

ITEM	QUICK REMINDER
HLD 7 Steps	Requirements → Scale → API Design → Data Model → HLD → Deep Dive → Bottlenecks
HLD Memory Trick	Really Serious APIs Don't Handle Bad Traffic
LLD 6 Steps	Entities → Relationships → Behaviors → Pattern → Interfaces → Java Code
LLD Memory Trick	Every Relationship Becomes A Pattern In Java
8 Blocks Memory	Load Databases Carefully. Make Cool APIs Often. Search Everything.
Decision Format	WHAT (choice) + WHY (evidence-based reason) + TRADE-OFF (what you give up)
Principal vs Senior	5 dimensions: Correctness, Reliability, Scalability, Maintainability, Cost
The 3 WHYs Rule	Never say 'use Redis.' Ask WHY 3 times until you reach physics or first principles.
Draw It Cold	After every session: blank paper, no notes, full diagram from memory. Find your gaps.
ADR Per Session	1-page Architecture Decision Record → push to GitHub docs/system-design/ after each session
The eBay Core Answer	Bids → Kafka → single consumer → DB (optimistic locking) → WebSocket to all bidders
Real Story Anchor	Twitter 2012 (fan-out). Stripe 2012 (idempotency). Netflix (microservices cost). Use these.